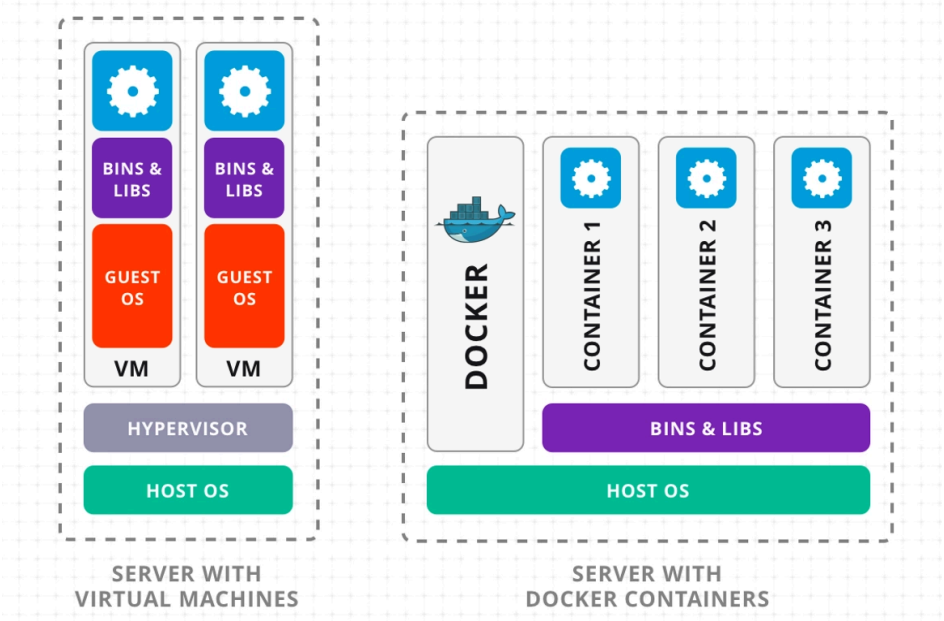
Notes based on https://www.youtube.com/watch?v=DQdB7wFEygo.
My GitHub repo is at https://github.com/Racekid16/docker-tutorial.

**What is Docker?**
Docker is a software development tool that helps you develop, ship, and run applications within lightweight containers. There are two essential terms to know:

*Image*- The recipe. Contains all the ingredients and instructions. This includes needed technologies, runtimes, and the tools/instructions to run the code.

*Container*- The actual meal that gets made from the recipe. One image can create multiple container instances.

Containers are useful because they make it easy to run code on different computers, since containers contain everything needed to run the code. Furthermore, they're isolated from the host and from other containers, preventing conflicts. Whereas virtual machines virtualize hardware, giving each VM its own OS kernel, containers share the host OS kernel while isolating processes.




**Installation**
Follow the instructions at https://docs.docker.com/desktop/features/wsl/ to install a Docker Desktop WSL 2 backend on Windows.

Run **`docker --version`** in the terminal to verify that Docker was successfully installed.

If you're using VS Code as your IDE, it's recommended to install the Docker extension because it has language support and other features that make developing with Docker easier.

**The Dockerfile**

To start, you need to create a Dockerfile, which is essentially the instructions to build the image. Here's an example Dockerfile for a simple express web server written in node.js.

```
FROM node:22

WORKDIR /app

COPY package*.json .

RUN npm install

COPY . .

ENV PORT=9000

EXPOSE 9000

CMD ["npm", "start"]
```

Explanation of each line:

- **`FROM node:22`** tells Docker to use the node.js version 22 Docker image as the base image for our image. It's the starting point. We use the node.js base image because the project is written in node.js, but there are others such as **`postgres:latest`** and **`ubuntu:latest`**.
- **`WORKDIR /app`** sets **`/app`** as the working directory of the container for all subsequent instructions in the Dockerfile. If the directory doesn't exist, Docker will create it automatically. Keep in mind that containers have their own file system that is separate from the host file system.
- **`COPY package*.json .`** copies **`package.json`** and **`package-lock.json`** from the working directory of the host to the working directory of the container.
- **`RUN npm install`** executes **`npm install`**, installing all the dependencies listed in **`package.json`** in the container's file system. Note that **`RUN`** commands happen during the image build process.

Note: usually in a node.js project, we get our code first and then install our dependencies, but here we install our dependencies first and then get our code. This is because Docker uses layer caching. You can think of each line in the Dockerfile as a layer. If a layer changed, rebuild it and all layers that come after it. Otherwise, use what's cached. Because of this, if we got our code first and then installed our dependencies, the image would re-install the dependencies every time our code changed. Thus, this ordering makes our build times faster.

- **`COPY . .`** copies all files, except those specified in **`.dockerignore`**, from the working directory of the host (specified by the first **`.`**) to the working directory of the container (specified by the second **`.`**).
- **`ENV PORT=9000`** sets an environment variable **`PORT`** to **`9000`** inside the container, which can be read at runtime (in the code, **`process.env.PORT`**).
- **`EXPOSE 9000`** documents that the container listens on port 9000. Note that this doesn't actually publish the port- that's done with the **`-p`** flag when running the container, or via the ports setting in **`compose.yaml`**.
- **`CMD ["npm", "start"]`** executes **`npm start`**, starting the container with the web server process. Note that the **`CMD`** command only runs when starting a container, not during the image build process, and you can only have one **`CMD`** command in a Dockerfile. Also note that if we instead wrote **`CMD npm start`**, the container would instead be started as a child process of the shell, which is more inconvenient to stop.

You can automatically generate a default **`Dockerfile`**, **`compose.yaml`**, **`.dockerignore`**, and **`README.Docker.md`** by running **`docker init`**.

**Building the image**
To build the image, run:

```
docker build -t <image_name> .
```

This uses the Dockerfile in the current directory (specified by **`.`**), and assigns the image the given name. It’s normal for the image build process to take a long time, even hours, if the project is complex.

**Starting and stopping a container**
For the express web server, we can start a container created from the image we built earlier with:

```
docker run -p 9000:9000 <image_name>
```

You can add the **`-d`** flag to start the container in detached mode.

You can add **`--name <container_name>`** to specify the name of the created container.

**`-p 9000:9000`** here maps the host’s port 9000 (first **`9000`**) to the container’s port 9000 (second **`9000`**), making the process running on port 9000 of the container accessible via port 9000 of the host.

To list all currently running containers, run (in a separate terminal, if needed):

```
docker ps
```

You'll see that all Docker containers have names which are randomly assigned if not specified. You can use a container's name to stop it by running (in a separate terminal, if needed):

```
docker stop <container_name>
```

**Docker Compose and Docker Volumes**

If we wanted to add a frontend and database to our project, it's not recommended that we put it all in one container, but instead give the frontend, backend, and database each their own containers, thus making a multi-container application.

One issue that arises from multi-container applications is that we now need to start and stop multiple containers each time we want to run our application, which can be cumbersome if done individually. Instead, we can use Docker Compose to tell our containers how to work together to start/stop the full application by creating a **`compose.yaml`** file.

Another issue with containers in general is that their data isn't persistent, so we'll lose it all once the containers stop. We can address this issue by using a Docker Volume, which is basically persistent Docker-managed storage that the containers can read and write from. We can create a Docker Volume by instructing Docker Compose to create it in **`compose.yaml`**, and tell the containers where they can access the Docker Volume.

Here's an example **`compose.yaml`** file that will start two containers, **`backend`** and **`db`**, and a volume **`postgres_data`** that **`db`** can access.

```yaml
services:
  backend:
    build: .
    ports:
      - '9000:9000'
  db:
    image: postgres:latest
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

To start the containers in **`compose.yaml`**, run:

```
docker compose up
```

You can add the **`-d`** flag to start the containers in detached mode.

To stop the containers in **`compose.yaml`**, run:

```
docker compose down
```

**Further study**

Consider looking into:

- Viewing information about containers in Docker Desktop
- **`docker scout quickview`**
- Docker Build Cloud